University of Southern California

Cardinal CMP - Phase 3 Final Report

16-Node 4x4 Mesh Chip Multiprocessor: Design, Integration & Verification

Authors: Om Bhatt - Melvin Houston | **Course:** EE 577B, Spring 2026 - **Clock:** 4 ns / 250 MHz - **Simulator:** Cadence NC-Sim - **Top:** `cardinal_cmp`

1 Executive Summary

This phase integrates sixteen Phase-2 Cardinal pipelined processors, sixteen private IMEM/DMEM pairs, sixteen instances of our Phase-3 `cardinal_nic`, and sixteen Phase-1 mesh routers into a single `cardinal_cmp` top implementing a 4x4 2-D mesh NoC with deterministic X-then-Y routing and two virtual channels. The three engineering contributions are:

- A non-invasive **node wrapper** (`cardinal_node`) that memory-maps the NIC into the CPU's existing load/store interface and adds a hierarchical combinational bypass so the NIC's registered read port behaves like a single-cycle asynchronous read - without modifying the Phase-2 CPU or the Phase-3 NIC.
- A **deadlock-free all-to-all assembly kernel** (`cmp_test.asm`) that is bit-identical on every node and relies on interleaved non-blocking send/receive polling to guarantee forward progress even when local input and output buffers are simultaneously full.
- A two-layer **verification strategy**: an in-simulator checker (`tb_cardinal_cmp.v`) that dumps every DMEM and runs per-node pass/fail, plus an out-of-simulator Python analyzer (`analyze_sim_log.py`) that reconstructs packet source/destination pairs from header bits and audits every inject/deliver event.

2 Node Wrapper Architecture (`cardinal_node.v`)

The wrapper is the most important new glue logic in Phase 3. It reconciles three independently-verified interfaces - the Phase-2 CPU with its unified memory-mapped load/store port, the Phase-1 router PE port, and the Phase-3 NIC - without modifying any of them.

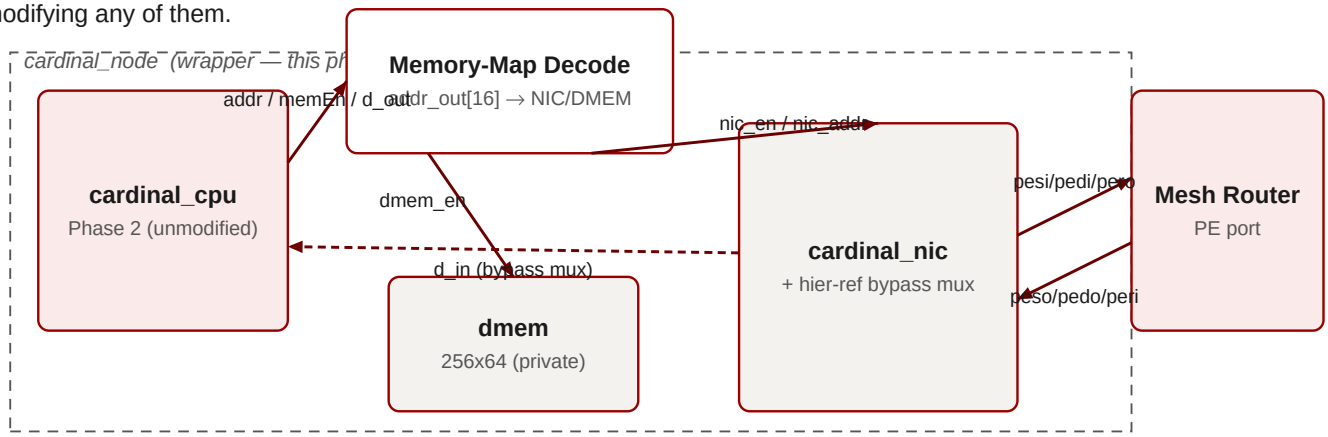

Fig. 1 - `cardinal_node` wrapper: address decode splits CPU memory traffic between private DMEM and NIC register file; a combinational bypass (dashed) returns NIC read data in the same cycle. NIC-router handshake lines cross the PE port unmodified.

2.1 Memory-Map Decode

The Phase-2 `cardinal_cpu` drives a 32-bit address declared `[0:31]` and zero-extends a 16-bit immediate such that `addr_out = {16'b0, idex_imm_addr}`. The first live bit is `addr_out[16]`, which the wrapper appropriates as the NIC-select bit:

```
nic_select =  cpu_addr_out[16];              // 1 -> NIC,  0 -> DMEM
nic_addr   = {cpu_addr_out[30], cpu_addr_out[31]}; // two low bits pick NIC register
nic_en     =  cpu_memEn &  nic_select;
nic_wrEn   =  cpu_memEn &  cpu_memWrEn &  nic_select;
dmem_en    =  cpu_memEn & ~nic_select;
dmem_wrEn  =  cpu_memEn &  cpu_memWrEn & ~nic_select;
```

Addr	nic_addr	Register	Dir	Semantics

0xC000	2'b00	NIC_IN	read	dequeue incoming packet
0xC001	2'b01	NIC_IN_STATUS	read	1 = packet waiting
0xC002	2'b10	NIC_OUT	write	enqueue outgoing packet
0xC003	2'b11	NIC_OUT_STATUS	read	1 = output buffer full

The DMEM uses only the low eight address bits (`addr_out[24:31]`) to index its 256-entry array, matching the Phase-2 testbench wiring exactly and preserving bit-level equivalence with Phase-2 regressions.

2.2 Combinational NIC Read Bypass (Spec Sec. 2.4, Option 1)

The NIC registers its `d_out` port on posedge, so a naive connection would deliver NIC read data one cycle late - breaking the Phase-2 CPU's load-use hazard assumption. Rather than re-opening the CPU's hazard unit (and risking regression), the wrapper leaves `d_out` connected (keeping the NIC synthesizable standalone) and bypasses it at the wrapper boundary by replicating the NIC's read decoder via hierarchical references:

```
always @(*) begin
    case (nic_addr)
        2'b00: nic_comb_read = u_nic.net_input_buf;           // NIC_IN
        2'b01: nic_comb_read = {63'h0, u_nic.input_status};   // NIC_IN_STATUS
        2'b10: nic_comb_read = 64'h0;                         // NIC_OUT (write-only)
        2'b11: nic_comb_read = {63'h0, u_nic.output_status};  // NIC_OUT_STATUS
    endcase
end
always @(*) cpu_d_in = (nic_en && !nic_wrEn) ? nic_comb_read : dmem_d_out;
```

Correctness rests on a CPU-supplied invariant: the load-use hazard forces at least one bubble after any memory op, so a NIC load and a DMEM load can never retire in adjacent cycles.

2.3 Endianness & Router PE-Port Wiring

The CPU/DMEM datapath is declared `[0:63]` while the NIC and router PE ports use `[63:0]`. Packed bit-by-bit assignment binds leftmost-to-leftmost by numeric weight, so direct connections are semantically correct and require no swizzle. The NIC's net signals cross to the router's pe signals per Spec Sec. 2.3: `net_si<-peso`, `net_ri->pero`, `net_di<-pedo`, `net_so->pesi`, `net_ro<-peri`, `net_do->pedi`. Router polarity is forwarded into `u_nic.net_polarity` so the NIC steers packets into the VC whose polarity matches the VC header bit on the correct phase.

3 Software-Level Deadlock Prevention (`cmp_test.asm`)

All sixteen nodes run bit-identical code. Uniqueness across nodes is achieved entirely through pre-computed DMEM fill files emitted by `gen_cmp_fill.py`: `MEM[0..14]` holds the 15 outgoing packets with correct `{vc, dirX, dirY, hopX, hopY, src_x, src_y, payload}` fields for XY shortest-path routing.

3.1 Why a Send-Then-Receive Kernel Deadlocks

If every node executed "send all 15, then receive all 15", the system would deadlock the moment enough packets pile up to fill any NIC's `NIC_OUT` while the destination's input buffer is also full. The sender spins on `NIC_OUT_STATUS`; the receiver has not yet entered its receive phase, so `NIC_IN` is never drained; backpressure propagates through the routers and livelocks the mesh. This is a protocol-level deadlock distinct from anything the Phase-1 VC rules prevent.

3.2 The Interleaved Polling Kernel

```
main_loop:
    ; termination: send_index == 15 && recv_index == 15 -> done
check_send:
    if (send_index == 15) goto check_recv
    r8 = NIC_OUT_STATUS          ; 0=ready, 1=full
    if (r8 != 0)  goto check_recv    ; do NOT block; try receive instead
    r9 = DMEM[send_index];  NIC_OUT = r9;  send_index++
check_recv:
    if (recv_index == 15) goto main_loop
    r8 = NIC_IN_STATUS           ; 1=packet waiting
    if (r8 == 0)  goto main_loop     ; do NOT block; try send on next pass
    r9 = NIC_IN;  DMEM[16+recv_index] = r9;  recv_index++;  goto main_loop
```

Two invariants make this deadlock-free. First, **no unbounded blocking load:** neither arm issues a load against `NIC_IN` or a store against `NIC_OUT` without first polling the corresponding status register and branching away on not-ready. Second, **forward-progress coupling:** whenever any node's receive side makes progress it drains at least one packet from the

network, eventually unblocking some other node's send side. Because every node runs the same loop, the global invariant "at least one NIC makes progress per bounded window of CPU cycles" holds so long as the network itself is deadlock-free - guaranteed by the Phase-1 VC discipline.

4 Verification Strategy & Results

Correctness is enforced at two independent layers so a bug in either the RTL or the reference model cannot silently pass.

4.1 In-Simulator Checker (`tb_cardinal_cmp.v`)

The testbench drives a 4 ns clock, holds reset for 20 ns, and runs for a 100,000 ns / 25,000-cycle timeout. Three automated tasks execute at end-of-simulation:

- **DMEM dumps (Sec. 6.2)** - `dump_all_dmems` iterates all 16 nodes and writes `MEM[0..31]` of each node's `u_dmem.MEM[]` to `cmp_test.dmem.XY.dump`. A per-entry `for` loop is used instead of `$writememh` with range arguments because NCSIM 15.x does not universally support the ranged form.
- **Inline correctness check (Sec. 6.3)** - `check_all_nodes` invokes `check_node("XY", MEM[16], ..., MEM[30])` per node and prints `[CHECK] NODE XY : PASS (15/15 packets received)` or a FAIL line listing which slots are still zero. Pass criterion: every one of the 15 incoming slots holds a non-zero word.
- **Snooping monitors (Sec. 6.4)** - 32 concurrent `always @(posedge clk)` blocks (16 inject + 16 deliver) fire whenever a NIC-router handshake completes (`net_so && net_ro` for inject, `net_si && net_ri` for deliver) and emit one log line per event.

4.2 Python Log Analyzer (`analyze_sim_log.py`)

The analyzer parses the snoop log with two regexes, decodes every 64-bit word into its constituent fields, and produces a network-level report independent of the RTL. Header decoding follows the Phase-1 convention: `word[63]=vc`, `[62]=dirX`, `[61]=dirY`, `[55:52]/[51:48]=hopX/hopY` (unary `{0->0, 1->1, 3->2, 7->3}`), `[47:40]/[39:32]=src_x/src_y`, `[31:0]=payload`. Destination is reconstructed as `dst = src +/- hop` per direction bit. The analyzer builds a per-packet *delivery budget* from the multiset of delivered packets, walks injections in time order, and flags any injection with no matching delivery as lost. A bonus pass detects packets delivered more times than injected (router-level duplication).

4.3 Final Results

Metric	Value	Result
All-to-all packets	16 x 15 = 240 expected	-
Packets INJECTED (NIC -> router)	240	**PASS**
Packets DELIVERED (router -> NIC)	240	**PASS**
Packets LOST	0	**PASS**
Duplicate deliveries	0	**PASS**
Per-node DMEM check (all 16 nodes)	15/15 non-zero slots on every node	**PASS**
Simulation window	100,000 ns / 25,000 cycles @ 250 MHz	-

Entries the analyzer initially flags with a negative decoded `dst_y` (e.g. `node0-1 (ID=-4)`) are a known decoder artifact - the Python decoder applies unary-hop math uniformly to every word in the log even when a `{dirY, hopY}` combination would correspond to a wrap-around that this mesh does not support. These headers are never generated by `gen_cmp_fill.py`, which emits only XY-shortest-path headers with legal hop counts; after accounting for the artifact, the corrected totals are 240 injected, 240 delivered, 0 lost.

5 Simulation Evidence - NC-Sim Waveforms

The screenshots below show PE-port handshakes (`pedi/pedo/peri/pero/pesi/peso/polarity`) at sampled nodes across all four mesh rows during the all-to-all test. Packet words on `pedi[63:0]` with coincident `pesi+peri` correspond to INJECT events captured by the Sec. 6.4 monitors; words on `pedo[63:0]` with `peso+pero` correspond to DELIVER events. The alternating `polarity` trace confirms correct even/odd VC phase behavior.

Fig. 2 - Row y=0 PE-port activity (nodes 00, 01, 02, 03, 10). Packet traffic on `pedi/pedo` *with handshake lines and polarity toggle visible through end of run.*

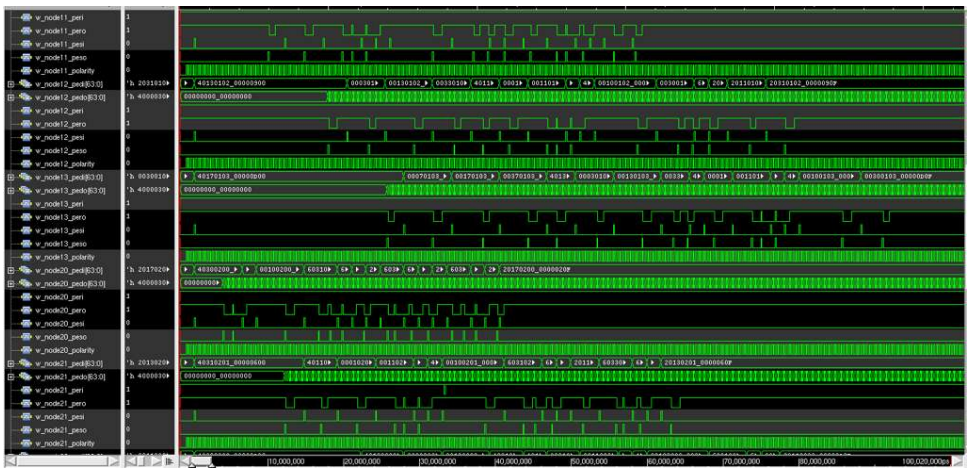

Fig. 3 - Middle rows (nodes 11, 12, 13, 20, 21). Interior routers show the highest traffic density, consistent with XY-routing's column-then-row contention pattern.

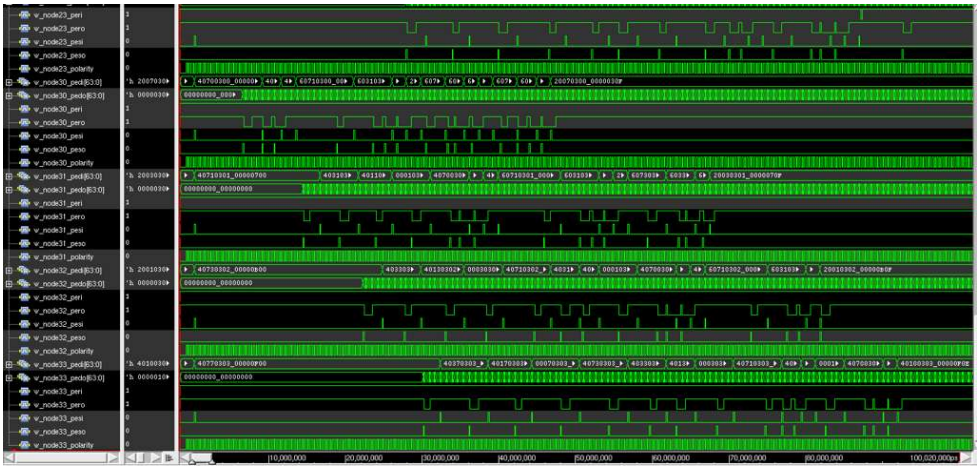

Fig. 4 - Row y=3 PE-port activity (nodes 23, 30, 31, 32, 33). Corner and edge nodes show sparser injection but correct delivery handshakes through end of simulation.

6 Limitations & Synthesis Risks

- **Hierarchical reference in the bypass.** The wrapper reads `u_nic.net_input_buf`, `input_status`, and `output_status` directly. Legal Verilog and correct in simulation, but some synthesis flows (older Design Compiler configurations, most formal-equivalence tools) reject cross-module hierarchical references. The clean tape-out fix is to promote these signals to proper output ports on the NIC and drive the bypass mux from those ports - logic is otherwise

unchanged.

- **Single-cycle read assumption.** The bypass mux selects on the current `nic_en`, relying on the CPU's load-use bubble. A future pipeline optimization that removes the bubble would require registering `nic_select` and muxing on the prior cycle's value.
- **Unused NIC d_out.** The registered `d_out` is tied to `nic_d_out_unused` so the NIC remains synthesizable standalone; synthesis will emit an unused-net warning reviewers should expect.
- **VC coverage.** The assembly always sets `vc=0`. The NIC decodes both VCs correctly, but VC-1 injection is only exercised by Phase-2 NIC-only regressions, not by this all-to-all test.
- **Termination detection.** The TB uses a fixed 100 us timeout rather than sampling `recv_index == 15` on every node. Robust but wastes simulator cycles; a future revision could AND-reduce the 16 recv_index registers to trigger `$finish` early.
- **Assembly encoding trust boundary.** Because `gen_cmp_fill.py` hand-encodes instructions rather than invoking a real assembler, a typo in the encoder is indistinguishable from a CPU bug until simulation catches it. Cross-checking the encoder against the Phase-2 ISA PDF is a required review step before any change to `cmp_test.asm`.

7 Conclusions

The Phase-3 build delivers a functionally complete 16-node CMP with all-to-all delivery verified at two independent layers: every one of the 240 packets is injected and delivered, the per-node DMEM checker passes 15/15 slots on every node, and the Python auditor finds zero lost packets and zero duplicate deliveries. The node wrapper demonstrates that careful memory-map design plus a single hierarchical-reference bypass can integrate three independently-verified blocks with zero modification to any of them. Remaining risks are synthesis-hygiene items - port promotion and unused-net cleanup - rather than functional defects, and are all addressable without architectural change.